AI WITH MUDIT

BEGINNER WORKBOOK
V3 / HAND HELD EDITION
UPDATED 2026

EDIT AND ANIMATE VIDEO WITH AI, FROM ZERO

Remotion for builders

Open the terminal, install the tools, point an AI agent at a raw clip, and ship a finished reel. No coding background needed. You direct, the agent writes the code.

FOR
Total beginners. Zero terminal, zero React, zero Remotion.

YOU BUILD WITH
Claude Code or Codex. The agent edits, you review.

YOU FINISH WITH
One real reel: captions, animation, clean HQ render.

A practical, come back to it guide.
Read one page, do one step,
then come back for the next.

DM "REMOTION"

START HERE

How to actually use this guide

This is not a theory PDF. It is a checklist you keep open while you build your first reel. One page is one step. Do the step, then come back for the next one.

1. Set up your machine → 2. Make one clip → 3. Preview → 4. Review → 5. Revise → 6. Render HQ

Jump to the page you need

NEVER OPENED A TERMINAL

Start at Part 1. Install Node, learn the terminal, install your agent. Pages 6 to 12.

HOW REMOTION THINKS

Frames, components, the timeline in code. Part 2, pages 13 to 19.

TELLING THE AGENT WHAT TO DO

Real prompts, the edit loop, templates. Part 3, pages 20 to 26.

CAPTIONS, ANIMATION, RENDER

The craft and the final export. Parts 4 and 5, pages 27 to 41.

The beginner promise

By the end you will not know every Remotion API. You do not need to. You will know what to ask the agent, which files to check, how to judge a preview like a viewer, and how to render the final clips. That is enough to ship.

DO THIS NOW

Pick **one** goal before you start: captions only, one animation, one split screen, or one CTA card. Do not try to build everything at once. The whole guide assumes you are working on one small thing.

READ THIS BEFORE YOU PANIC ABOUT CODE

You are the director. The agent writes the code.

Remotion makes videos out of React code. That sounds scary. Here is the part nobody tells beginners: you are not the one writing that code. The AI agent is.

YOUR JOB

- **Taste.** Decide what looks good and what does not.
- **Direction.** Say exactly what each clip should do.
- **Review.** Watch the preview like a normal viewer.
- **Judgement.** Approve, or ask for one specific fix.

THE AGENT'S JOB

- **Write the React.** All the Remotion code.
- **Place the clips,** captions, animations, timing.
- **Run the render** and hand you a preview file.
- **Make the fix** you asked for, nothing more.

> **You do not need to learn React or JavaScript to finish this guide.** You need to read a preview honestly and write clear instructions. If you can describe what is wrong in plain words, the agent can fix it. That skill matters more than any API.

So why learn any of this Remotion stuff?

Because a director who understands frames, layers, and safe zones gives sharper instructions and catches mistakes faster. You are learning just enough of the machine to point it well. Not enough to operate it by hand.

DO THIS NOW Drop the goal of "becoming a coder." Adopt the goal of "becoming the person who can describe a good edit in one paragraph." That is the real skill this guide builds.

THE ONE PICTURE TO REMEMBER

The whole loop, start to finish

Every reel you ever make with this stack runs through the same six steps. Memorise this loop and nothing in the rest of the guide will feel random.

1. **You hand over the assets**
   Raw clip, the exact words spoken, and your rules. The agent now has what it needs.
2. **You write one clear prompt**
   Edit this one clip. Captions exact. One visual idea. Save a preview and a short report.
3. **The agent edits the code and renders a preview**
   It writes the Remotion code for you, then exports a small, fast video file to review.
4. **You watch it like a viewer**
   Is the hook clear? Captions right? Face visible? Can you read the CTA? Be honest.
5. **You ask for one specific fix, or approve**
   "The 3 looks like an S, redraw it." One change at a time. Not a vague redo.
6. **You lock the version and render HQ**
   Once approved, render that clip at full quality. Only then move to the next clip.

DO THIS NOW

Notice that steps 1, 2, 4, 5, and 6 are **you**. Only step 3 is the agent. This guide spends most of its time teaching you to do your five steps well.

WHAT LIVES ON YOUR MACHINE

The six tools you need

You install these once. After that, every reel is just a new folder using the same setup. Do not buy or download anything else yet.

1. Node.js

The engine Remotion runs on. Free. You install it first because nothing else works without it.

2. The terminal

The text window where you type commands. Already on your computer. You just learn to open it.

3. VS Code

A free code editor. It is your project's home, and it has a terminal built in so you stay in one window.

4. Claude Code or Codex

Your AI agent. It reads your files and writes the video code. This is the tool that actually does the editing.

5. Remotion

The library that turns code into video. You create it inside your project with one command. No separate install.

6. Your raw clips

Your face cam footage and a text file of the exact words you said. The agent needs both to caption correctly.

Two things cost money, four are free

Node, the terminal, VS Code, and Remotion are free. Your AI agent needs a paid plan to run (Claude Code needs a Claude Pro or higher plan, Codex needs an OpenAI plan). That is the only real cost to start.

DO THIS NOW Do not install everything at once in a panic. Turn the page and do Step 1 only. We go one tool at a time.

PART 01

Set up your machine

Six small installs. Do them in order. After this page you have a working setup that every future reel reuses.

06 Step 1 / Install Node.js

07 Step 2 / Meet the terminal

08 Step 3 / Install VS Code

09 Step 4 / Install your AI agent

10 Step 5 / Create your first project

11 Step 6 / Open Studio and the agent

12 When something breaks

STEP 1 / THE ENGINE

Install Node.js

Node.js is the engine that runs Remotion. Install it first. If you skip this, every later command fails with "command not found."

MACOS & WINDOWS

- Go to **nodejs.org** in your browser.
- Download the **LTS** version (the big green button, not "Current").
- Open the file and click Next through the installer. Defaults are fine.
- Restart your computer once. This sets the path correctly.

WHY LTS, NOT CURRENT?

LTS means Long Term Support. It is the stable version that tools are tested against. "Current" is the bleeding edge and breaks more often. As a beginner you always want LTS.

NOW CONFIRM IT WORKED

Open your terminal (next page shows you how) and type these two lines, pressing Enter after each.

terminal

```
$ node -v
$ npm -v
```

WHAT YOU SHOULD SEE

```
v22.14.0 10.9.2 # two version numbers. The exact numbers will differ. # Anything that starts with "v" and a number means it worked.
```

If you see `command not found` instead, Node did not install or your terminal needs a restart. Close the terminal fully, reopen it, and try again. Still stuck? Page 12.

DO THIS NOW

Install Node LTS and run **node -v**. Do not move on until you see a version number. This is the foundation. Everything else sits on top of it.

STEP 2 / THE COMMAND WINDOW

Meet the terminal

The terminal is just a window where you type commands instead of clicking buttons. It looks intimidating. It is not. You only need four commands to start.

MACOS

Press `Cmd` + `Space`, type **Terminal**, hit Enter. A black or white window opens. That is it.

WINDOWS

Click Start, type **PowerShell**, hit Enter. Use PowerShell, not the old black Command Prompt.

THE ONLY FOUR COMMANDS YOU NEED RIGHT NOW

`cd folder-name`
Move into a folder. Short for "change directory."

`cd ..`
Move back out, up one folder.

`ls` (mac) `dir` (win)
List what is inside the current folder.

`clear` (or cls)
Wipe the screen clean when it gets messy.

READING THE PROMPT

```
mudit@macbook ~/projects $ your command goes here # The bit before the $ tells you who you are and where you are. # You type after the $. When this guide shows "$", do not type the $ itself.
```

One habit that saves you: the terminal is always "inside" one folder. Before you run a project command, make sure you have used `cd` to move into the right project folder first. Half of all beginner errors are just being in the wrong folder.

DO THIS NOW Open your terminal. Type **ls** (mac) or **dir** (windows) and press Enter. You just ran your first command. Nothing broke. Good.

STEP 3 / YOUR PROJECT HOME

Install VS Code

VS Code is a free editor from Microsoft. You will not write code in it by hand, but it is where your project lives, where you watch files change, and it has a terminal built right in so you stay in one window.

1. **Download it**

   Go to **code.visualstudio.com**, download for your system, install with default settings.

2. **Open a folder, not a file**

   This is the key habit. File menu, then Open Folder, and pick your project folder. VS Code always works on a whole folder.

3. **Open the built in terminal**

   Use the menu: View, then Terminal. A terminal opens at the bottom, already inside your project folder. No more `cd` hunting.

Why this matters more than it looks

When the built in terminal is open inside your project folder, you are always in the right place to run commands. And when the agent edits a file, you see it update live in the editor on the left. One window, full visibility.

> **Optional but nice:** in VS Code, open Extensions (the squares icon on the left) and you can later add a Remotion preview extension. Skip this for now. Default VS Code is more than enough to finish this guide.

DO THIS NOW Install VS Code. Create an empty folder somewhere easy to find, like **Documents/reels**. Open that folder in VS Code, then open the built in terminal. That terminal is now your home base.

STEP 4 / THE TOOL THAT DOES THE EDITING

Install your AI agent

This is the most important install. Claude Code or Codex is the agent that reads your files and writes the Remotion code. This guide uses Claude Code commands, Codex works the same way.

INSTALL CLAUDE CODE (RECOMMENDED PATH)

macOS / Linux terminal

```
$ curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell

```
> irm https://claude.ai/install.ps1 | iex
```

This native installer needs no extra setup. If you would rather use npm, run `npm install -g @anthropic-ai/claude-code` instead (needs the Node you installed in Step 1).

CONFIRM AND SIGN IN

- Run `claude --version` to confirm a version prints.
- Run `claude` inside a folder. First launch opens your browser to sign in.
- If anything looks off, run `claude doctor`. It checks the whole install for you.

USING CODEX INSTEAD

Prefer OpenAI's agent? Install the Codex CLI, sign in with your OpenAI account, and run `codex` in your project folder. Every prompt template in this guide works the same. Just type it to Codex instead of Claude.

Heads up on cost: the agent needs a paid plan to run. Claude Code requires a Claude Pro plan or higher. This is the one real cost in the whole setup. Everything else on these pages is free.

DO THIS NOW

Install your agent and run **claude --version**. When you see a version number and you have signed in, your machine is fully set up. The next page is the fun part: making a project.

STEP 5 / YOUR FIRST PROJECT

Create your first Remotion project

One command builds a complete, working Remotion project for you. You do not set anything up by hand. Run it inside your reels folder.

terminal / inside your reels folder

```
$ npx create-video@latest
# a menu appears. Use arrow keys. Pick "Hello World" to start.
# give it a name when asked, for example: my-first-reel

$ cd my-first-reel    # move into the new project folder
$ npm install         # only if it did not auto install
```

WHAT YOU SHOULD SEE

```
? Choose a template › ❯ Hello World Blank Hello World (TypeScript) # after it finishes:
Created project at my-first-reel. Run npm start to begin.
```

IF THE MENU CONFUSES YOU

Arrow keys move the cursor. Enter selects. Just pick the top option, Hello World. You can always make a fresh project later. Do not overthink the first choice.

WHAT JUST HAPPENED

Remotion created a full project: a few sample compositions, a public folder for your clips, and all the config. You now have a real video project on your machine.

DO THIS NOW Run **npx create-video@latest**, pick Hello World, and **cd** into the folder. Then open that same folder in VS Code (File, Open Folder). One folder, two windows on it.

STEP 6 / SEE IT MOVE, THEN POINT THE AGENT

Open Studio and the agent

Two things run side by side from now on: Remotion Studio (to watch your video) and your agent (to edit it). Open both from inside your project folder.

A / SEE THE VIDEO

terminal

```
$ npx remotion studio
```

Your browser opens Remotion Studio. You see the sample video and a play button. Press play. Code became video. That is the whole magic.

B / OPEN THE AGENT

a second terminal tab

```
$ claude
# or:  codex
```

Run this in a new terminal tab, still inside the project. The agent now sees your whole project and is ready for instructions.

Why two tabs: Studio keeps running and refreshing while you work. The agent runs separately so it can edit files. Leave Studio open in one tab, talk to the agent in the other. In VS Code you can split the bottom terminal into two.

You are now fully set up

Node, terminal, VS Code, agent, and a live Remotion project. From here the rest of the guide is about **directing**, not installing. If a command failed along the way, the next page fixes the common ones.

DO THIS NOW

Run **npx remotion studio**, press play on the sample video, then open **claude** in a second tab. Say hello to it. Confirm it replies. Setup complete.

SETUP FIRST AID

When something breaks

Every beginner hits at least one of these on day one. None of them mean you did anything wrong. Find your error message, do the fix, move on.

`command not found: node` or `claude`

The tool installed but your terminal is using an old session. **Fully close the terminal and reopen it.** On Mac, quit Terminal with Cmd+Q, do not just close the tab. Still missing? Restart the computer once.

`Node version too old` / EBADENGINE warnings

You have an old Node. Reinstall the latest **LTS** from nodejs.org over the top. Then check again with `node -v`. You want version 18 or higher.

`EACCES` / permission denied during install

Do **not** retype the command with `sudo`. That makes it worse. For Claude Code, use the native installer from Step 4 instead of npm. It sidesteps permissions entirely.

`Port 3000 already in use` when starting Studio

Studio is already running in another tab. Either use that one, or close the old tab. Worst case, close all terminals and run `npx remotion studio` fresh.

The agent says it `cannot find a file`

Two causes. Either the agent was not opened inside the project folder, or you gave the wrong filename. Close the agent, `cd` into the project, reopen it, and tell it the exact filename and folder.

> **The universal first move:** read the error out loud, copy the exact red text, and paste it to your agent with "I got this error, what do I do." The agent is good at fixing its own environment. Use it.

PART 02

How Remotion thinks

Just enough of the machine to point it well. Frames, components, layers, and the timeline written as code. You will not memorise APIs. You will build intuition.

13 What is Remotion, really

14 Not just an animation tool

15 Project anatomy in plain language

16 Folder structure that prevents chaos

17 Frame math cheat sheet

18 Sequence / the timeline in code

19 Importing clips, audio, images

THE CORE IDEA

What is Remotion, really

Simple answer: Remotion makes real videos out of code. You describe what should appear at every frame, and Remotion renders the final MP4.

> **Beginner translation.** A normal editor like CapCut means dragging clips on a timeline by hand. Remotion means writing instructions that say what appears at every moment. The agent writes those instructions for you.

FRAME

One moment of the video. At 60fps, frame 60 is one second in. Everything is timed in frames.

COMPONENT

A reusable visual block: a caption, a tag, a card, a split screen, an animation.

PROPS / DATA

The inputs that change the video: text, colours, clip paths, timings, names.

RENDER

The step where Remotion turns the code into a finished MP4 or other file.

THE MENTAL MODEL

`React component` + `frame number` + `props / data` → `video`

DO THIS NOW

In your project, open the file **Root.tsx** in VS Code. That file is the menu. It lists every video, called a composition, that Remotion can render. You do not need to understand the code yet, just find the file.

HONEST POSITIONING

Not just an animation tool

When someone asks "why not just use CapCut or Premiere," here is the real answer. Remotion wins when the edit has rules. A normal editor wins when it is one random manual cut.

✓ BEST FOR

- Repeatable reel templates
- Captions and timed overlays
- Data driven videos
- Motion graphics from code
- AI agent controlled editing

× NOT BEST FOR

- Dragging clips quickly by hand
- One off edits with no reusable logic
- Heavy manual masking and VFX
- Rescuing badly shot footage

The creator rule

If the edit has rules, patterns, timing, templates, or many similar outputs, Remotion becomes powerful. Your reel series, your guide CTAs, your repeated formats are exactly that. If it is a single random manual cut, a normal editor is faster. Use the right tool.

DO THIS NOW

Reach for Remotion when you want a **system**, not just a single edit. A repeatable format you will use fifty times is the perfect first project.

WHAT EACH FILE DOES

Project anatomy in plain language

A Remotion project looks technical, but each file has one simple job. You only need to recognise a few of them.

`Root.tsx`
The menu. Registers which videos can be rendered.

`Clip1.tsx`
One scene or one clip edit. You will have several.

`components/`
Reusable blocks: captions, tags, cards, transitions.

`public/`
Your raw video, audio, and image files live here.

`package.json`
Lists the tools and the commands the project uses.

`remotion.config.ts`
Project wide render and config settings.

WHAT A COMPOSITION LINE LOOKS LIKE

src/Root.tsx

```
// "Remotion, this is a video you can render"
<Composition
  id="Clip1"
  component={Clip1}
  durationInFrames={210} fps={60}
  width={1080} height={1920} />
```

DO THIS NOW When your agent creates a clip, ask it: "which composition id do I render for this?" Write that id in your report. You will need it for the render command later.

PREVENT THE MESS

Folder structure that prevents chaos

Most AI agent projects turn messy because outputs scatter everywhere. Use this structure every time and you will always know which version was approved.

project tree

```
my-first-reel/
  clip1.mp4  clip2.mp4
  transcript.md
  REEL_EDITING_RULES.md
  src/
    Root.tsx  Clip1.tsx
  public/
    clip1.mp4
  edited_outputs/
    day3_reel/
      previews/
      reports/
      final_individual_clips_hq/
```

FOLDER RULES

- **Raw clips:** never overwrite them.
- **Previews:** small files, just for review.
- **Reports:** what changed and what was assumed.
- **HQ renders:** approved clips only.

WHY THIS MATTERS

When you come back a week later, you should instantly know which file was final. No more "wait, which one did I post?" confusion.

DO THIS NOW

Create **REEL_EDITING_RULES.md** once. Put your recurring rules in it: exact captions, safe zones, one clip at a time, new output folder each time. Tell the agent to read it before every edit.

THE ONE BIT OF MATH

Frame math cheat sheet

Understand frames and Remotion stops feeling scary. This is the only arithmetic you need.

AT 60FPS

60 frames = 1 second

So frame 120 is 2 seconds, frame 288 is 4.8 seconds.

THE FORMULA

frame = seconds × fps

WORKED EXAMPLE

You want an animation to start at 1.5 seconds, at 60fps.

1.5 × 60 = frame 90. So you tell the agent: start this at frame 90.

FRAME THINKING IN CODE (THE AGENT WRITES THIS)

inside a component

```
const frame = useCurrentFrame();
const appears = frame >= 90;  // starts at 1.5s
const opacity = interpolate(frame, [90, 110], [0, 1], {
  extrapolateLeft: 'clamp', extrapolateRight: 'clamp',
});
```

DO THIS NOW

Write your timings in **seconds first**, then convert. "CTA appears at 4.8s" becomes frame 288 at 60fps. Give the agent the frame number and it places it exactly.

SHOW THIS, DURING THIS PART

Sequence: the timeline in code

A Sequence is how you say "show this layer during this part of the video." It is the timeline, written as code instead of dragged by hand.

- `from={120}` start this layer at frame 120.
- `durationInFrames={90}` keep it visible for 90 frames.
- **Layering:** video, captions, animations, tags, and CTA cards stack on top of each other.
- **Trimming:** use start and end timing to cut media down.

the recipe

```
<Sequence from={0} durationInFrames={120}>
  <ClipVideo />
</Sequence>

<Sequence from={50} durationInFrames={80}>
  <Caption text="AI se kya seekha?" />
</Sequence>
```

THINK OF EACH CLIP AS STACKED LAYERS

video | captions | animation | tags | CTA card | audio

DO THIS NOW

For every clip, list the layers **before** the agent codes anything: video layer, captions layer, animation layer, CTA layer, audio layer. A clear layer list makes a clear prompt.

WHERE YOUR MEDIA GOES

Importing clips, audio, images

Put your assets in the `public/` folder so Remotion always finds them reliably. Then the agent references them by exact filename.

`public/ folder` → `staticFile()` → `Video / Audio / Img` → `layer in scene` → `render`

COMMON ASSETS

- mp4 clips
- mp3 / wav
- png / jpg
- webp
- json captions
- fonts

BEGINNER RULE

Do not make the agent hunt randomly for files. Tell it exact filenames and exactly where outputs should go. Precision in, precision out.

the asset pattern

```
import {OffthreadVideo, staticFile, Audio, Img} from 'remotion';

<OffthreadVideo src={staticFile('clip1.mp4')} />
<Audio src={staticFile('music.mp3')} />
<Img src={staticFile('guide-cover.png')} />
```

DO THIS NOW

Before prompting the agent, make a tiny list: **clip1.mp4, transcript.md, approved preview, output folder.** Hand it that list. No guessing required.

PART 03

Direct the agent

The agent can edit because the video is code. But it edits well only when you instruct it well. This is where most of your skill lives.

20 How the agent actually edits video

21 Your very first prompt, step by step

22 The edit loop

23 Bad prompt vs good prompt

24 Template / edit one clip

25 Template / revise a weak preview

26 Template / render approved clips at HQ

BEHIND THE SCENES

How the agent actually edits video

You type an instruction. The agent does four things, every time. Once you see the pattern, you can predict and trust it.

1. **It reads your files**
   The transcript, your rules file, the existing clip code, and the raw video in public. It builds context before touching anything.

2. **It writes or edits the React code**
   It changes the clip's .tsx file: places the video, adds caption components, sets the animation timing in frames. You can watch the file change in VS Code.

3. **It runs the render command**
   It calls Remotion to export a preview MP4 to your output folder. It runs the terminal command for you, you do not type it.

4. **It writes a short report**
   What it changed, what it assumed, the composition id, the output path. This is your paper trail so nothing gets lost.

> **The agent asks before big changes.** It will show you the edit it wants to make and wait for you to approve. Read it, then say yes. If something looks wrong, say so before approving. You are always in control.

DO THIS NOW

Keep VS Code open on the left while the agent works. Watching the clip file change in real time turns this from "magic black box" into "oh, I see what it did." That understanding makes your next prompt sharper.

THE EXACT WORDS TO TYPE

Your very first prompt

Put one clip and one transcript in your project. Open the agent. Copy this, replace the words in brackets, and send it. This is a real, complete first prompt.

type this to claude or codex

```
> Edit only clip1. Read transcript.md for the exact words.
  Place clip1.mp4 full screen. Add captions in 2 to 4 word
  chunks near my chin, never over my mouth. Highlight the
  word "Remotion" in yellow. Render a preview to
  edited_outputs/day1/clip1_captions_preview_v1.mp4
  and write a short report next to it. Do not edit any
  other file. Show me your plan before you start.
```

WHAT THE AGENT DOES NEXT

1. Reads transcript.md and clip1.mp4 2. Shows a plan: "I'll create Clip1.tsx with a Caption..." 3. Asks: Approve this edit? (y/n) ← you type y 4. Writes the code, runs the render 5. ✓ preview saved to edited_outputs/day1/...

THEN YOU

Open that preview file and watch it. Like a viewer, not a coder. Hold the next page open while you judge it.

IF IT STALLS

If the agent says it cannot find a file, you are probably not inside the project folder, or the filename is off. Fix that and resend.

DO THIS NOW Actually send this prompt with one real clip. Approve the plan. Watch the preview. You just edited a video with code without writing a single line yourself.

REPEAT THIS FOR EVERY CLIP

The edit loop

This is the rhythm of every reel. Give assets, edit one clip, preview, review honestly, revise once, lock the version. Then the next clip.

`give assets` → `edit one clip` → `preview + report` → `review honestly` → `revise once` → `lock version`

The golden rule

Do not hand the agent the whole reel at once, unless you are doing the final merge. Edit clip by clip. Clip by clip editing gives you control and stops one bad effect from ruining the entire reel. One clip, approved, then the next.

Revise once, not five times. If a preview needs more than one or two rounds of fixes, your instruction was too vague. Stop, rewrite the prompt clearly, and start that clip fresh. Endless tiny tweaks waste more time than a clean restart.

DO THIS NOW

Before every prompt, write six things: input files, the exact spoken line, caption placement, the one visual idea, the output folder, and what **not** to edit. Those six lines are your prompt skeleton.

THE DIFFERENCE THAT DECIDES EVERYTHING

Bad prompt vs good prompt

The agent is only as good as your instruction. Vague prompts force it to guess captions, layout, timing, and output. Specific prompts get you what you pictured.

× BAD PROMPT

"Make this look viral and add good animations."

Too vague. The agent has to invent the captions, the layout, the timing, and where to save the file. You will hate the result and not know why.

✓ GOOD PROMPT

"Edit only clip3. Use a 36/64 split. Captions exact from transcript. Bottom is a mature Remotion canvas. Save a preview and a report."

Specific. One clip, one layout, exact captions, one clear visual, defined output. The agent has nothing to guess.

SPEAK IN VIEWER LANGUAGE, NOT CODE LANGUAGE

Say **"the 3 looks like a weird S, redraw it cleaner"** instead of "improve the number animation." A specific viewer complaint creates a specific fix. You do not need technical words. You need precise observations.

DO THIS NOW

Take any vague idea in your head and rewrite it as: one clip, one layout, exact words, one visual, one output path. If your prompt still says "make it nice," it is not ready.

COPY AND REUSE

Template / edit one clip

Use this when you want the agent to create a first preview. Replace every bracket before sending.

single clip prompt

```
Edit ONLY clip[number].
Use: clip[number].mp4, transcript.md, REEL_EDITING_RULES.md
Do not edit other clips. Do not merge the reel.
Create only a preview render and a report.

Clip purpose:
  [one sentence: hook / explanation / proof / CTA]

Captions: use the exact spoken words:
  "[paste the exact line]"
Caption placement: around chin, never over mouth, IG safe.

Visual idea:
  [one clear idea: number draw / split screen / CTA card]

Output:
  edited_outputs/[project]/clip[n]_[idea]_preview_v1.mp4
  edited_outputs/[project]/clip[n]_[idea]_report_v1.md
```

DO THIS NOW

Replace every bracket before you send it. If the prompt still contains **[something]**, it is not ready. Brackets left in are the number one beginner mistake here.

COPY AND REUSE

Template / revise a weak preview

The best revisions are specific. Say what works, what fails, and exactly what to change. Then protect everything that was already good.

revision prompt

```
Revise ONLY clip[number].
Use latest preview: [path/to/latest_preview.mp4]
Use latest report:  [path/to/latest_report.md]

What works (keep these):
  - [keep this]

What is weak:
  - [specific viewer problem]

Change only these things:
  1. [specific fix]
  2. [specific fix]

Do not change: captions, approved layout, audio,
other clips.

Save as:
  edited_outputs/[project]/clip[n]_[idea]_preview_v2.mp4
```

Use viewer language. "The 3 looks like a weird S" beats "improve the number animation" every time. Specific viewer problems produce better fixes than abstract requests.

DO THIS NOW

List what to keep **before** you list what to change. The "do not change" block is what stops the agent from breaking the parts you already liked.

COPY AND REUSE

Template / render approved clips at HQ

Use this only after every individual preview is approved. Render each clip at full quality, still separate, do not merge yet.

HQ render prompt

```
Render each approved clip individually at highest quality.
Do NOT merge the reel yet. Do NOT re-encode preview files.
Render from the Remotion compositions.

Approved versions:
  Clip1: [composition / preview name]
  Clip2: [composition / preview name]

Output folder:
  edited_outputs/[project]/final_individual_clips_hq/

Render settings:
  1080x1920, 60fps if compositions are 60fps,
  h264, CRF 14 or 16, yuv420p, clean AAC audio.

Write HQ_RENDER_REPORT.md with paths, duration, codec,
fps, size, and confirm the reel was NOT merged.
```

DO THIS NOW

Do not ask for the final merge until you have watched every HQ individual clip. Final merge is the **last** step, not the cleanup step. Watch first, merge last.

PART 04

The craft

Now the taste. Captions that read, animation that looks premium not childish, safe zones that survive Instagram, and audio that keeps your voice on top.

27 Recipe / edit one face cam clip

28 Captions workflow you reuse

29 Instagram safe zone layout

30 Animation / simple but premium

31 Mature motion graphics

32 Split screen / face plus proof

33 Audio that keeps your voice on top

THE RECIPE BEHIND MOST REELS

Recipe / edit one face cam clip

This is the base most of your reels are built on. Five moves, in order, every time.

`read transcript` → `place video` → `exact captions` → `one visual idea` → `preview + report`

CLIP CHECKLIST

- **Captions:** exact words, 2 to 5 per chunk, placed at chin or upper chest.
- **Face:** eyes and mouth visible. Never cover the expression.
- **Visual:** one main idea only, a number draw, a tag, a split screen, or a CTA card.
- **Safe zone:** avoid the bottom 250px and the far right UI strip.

> **The biggest beginner mistake:** trying to make every second visually crazy. A reel needs hierarchy. Hook, explanation, proof, CTA. Each clip has one job. Crowding kills the message.

DO THIS NOW

For the next clip you edit, write the clip's purpose in **one sentence** before you ask the agent to touch the video. If you cannot say its job in a line, you are not ready to edit it.

PART OF THE EDIT, NOT SUBTITLES

Captions workflow you reuse

Captions should feel like part of the edit, not random subtitles slapped on top. Get these four rules right and the reel instantly looks more pro.

CAPTION RULES

- **Exact:** match what is actually spoken.
- **Short chunks:** 2 to 5 words when possible.
- **Position:** chin or upper chest, never over the mouth.
- **Highlight:** one important word in yellow.

ALWAYS FIX BY HAND

Auto transcription mishears technical words. Check these every time:

- Remotion / emotion
- Claude / cloud
- Codex spelling
- DM keyword

caption brief you give the agent

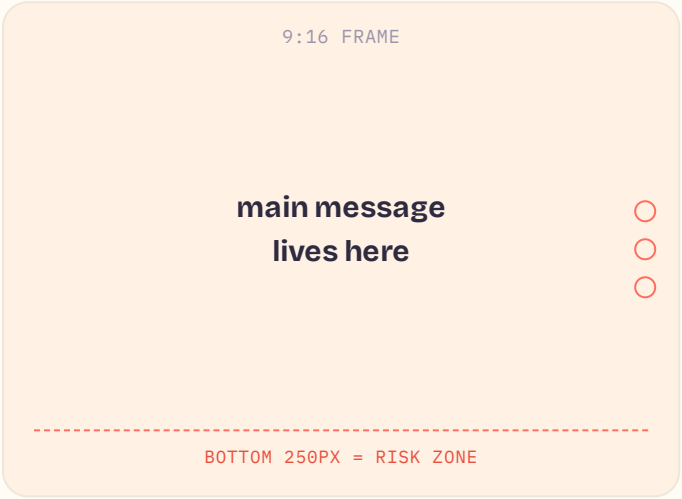
```
Caption chunks for this clip:
  1. "Ek example main dikha raha hoon"
  2. "yeh jo possible hota hai,"
  3. "yeh hota hai Remotion ki wajah se."
Highlight: Remotion
Position: chin / upper chest, not mouth
```

DO THIS NOW After transcription, read the captions out loud once. If a caption sounds like a robot rewrite of what you said, fix it **before** editing. Your real words always beat the auto version.

SURVIVE THE INSTAGRAM UI

Instagram safe zone layout

Your edit can look perfect in preview and still fail on Instagram, because the app's buttons cover your important text. Plan around them.



SAFE RULES

- Main text centre left.
- CTA above the bottom 250px.
- Avoid the far right edge where buttons sit.
- Captions near the chin, proof in the middle.

YOUR RULE

If a viewer cannot read the CTA because of buttons, the edit failed, even if the animation is gorgeous.

DO THIS NOW Before you approve any preview, picture the Instagram buttons on the right and the caption bar on the bottom. Then ask: is the key message still readable? If not, send it back.

NUMBERS CHANGING SMOOTHLY

Animation / simple but premium

Most good Remotion animation is just numbers changing smoothly over frames. You do not need five effects. You need one clean one.

THE FOUR PROPERTIES BEGINNERS USE MOST

opacity
fade in and out

translateX / Y
move across the frame

scale
pop or push toward camera

rotate
spin or tilt

the core pattern the agent writes

```
const frame = useCurrentFrame();
const opacity = interpolate(frame, [0, 20], [0, 1], {
  extrapolateRight: 'clamp',
});
const scale = spring({frame, fps, from: 0.9, to: 1});
```

DO THIS NOW

Start every animation with one move: a fade plus a small scale. If it already feels clear, stop. Do not add five more effects to prove you can.

LOOK LIKE A BUILDER, NOT A BIRTHDAY CARD

Mature motion graphics

For AI and builder content, mature motion beats childish animation. The words you put in your prompt decide which one you get.

✓ SAY THIS IN PROMPTS

- dark grid
- code chips
- motion paths
- keyframe dots
- procedural particles
- kinetic typography

× AVOID THESE WORDS

- cute rocket
- fire
- stick figure
- random cartoon
- confetti blast

THE VOCABULARY THAT READS PREMIUM

Code chips showing `useCurrentFrame()`, an orbital system, motion paths, kinetic type that snaps in on the beat. Clean, technical, restrained. That is the visual language of someone who builds, which is exactly what your audience trusts.

DO THIS NOW

Put the mature words directly in your prompt. The agent matches your vocabulary. Ask for "a dark grid with motion paths," not "cool animation," and you get the premium version.

FACE PLUS PROOF AT ONCE

Split screen / face plus proof

Use split screen when the viewer needs to see your face and the proof at the same time. The ratio you choose decides how it feels.

GOOD RATIOS

- **36 / 64:** face visible, big demo. The default.
- **40 / 60:** more face, less demo.
- **50 / 50:** balanced, but can waste space.

FOR INSTAGRAM

Keep the main proof in the upper or middle of the bottom canvas. Do not put the final text at the very bottom where the caption bar covers it.

layout brief

```
Top:     36% real video
Bottom:  64% Remotion canvas
Divider: yellow line
Labels:  small only
```

DO THIS NOW

When you use split screen, write the ratio in the prompt. Example: "Use 36/64, top video, bottom Remotion canvas." Naming the ratio removes the guesswork.

VOICE STAYS KING

Audio that keeps your voice on top

Audio matters, but for face cam reels your voice must stay the priority. Music supports it, it does not fight it.

`voice` → `music low` → `SFX rare` → `loudness check` → `render`

RULES

- **Voice first:** clean, audible, no distortion.
- **Music lower:** background never fights the words.
- **SFX rare:** subtle UI hits only, no cheap whooshes.
- **Sync:** animation hits land on words like "DM" or "Remotion."

the volume idea

```
<Audio src={staticFile('music.mp3')}
  volume={0.12} />
<OffthreadVideo src=
{staticFile('clip1.mp4')}
  volume={1} />
```

DO THIS NOW

For educational reels, do not add loud SFX during important lines. If viewers miss the words, the edit failed, no matter how good the music drop felt.

PART 05

Ship it

The clip is good in preview. Now lock it, render it clean, and post it. This is the part most beginners rush. Slow down here and the reel looks professional.

34 Preview vs HQ render

35 Your 4 clip reel blueprint

36 Common mistakes and fixes

37 Final reel checklist

38 Your 5 day practice plan

39 Quick command cheat sheet

40 Source notes and docs

TWO KINDS OF RENDER

Preview first, HQ only when approved

Do not waste time rendering at high quality until the clip is approved. Preview renders are fast and a bit rough. HQ renders are slow and clean. Get the clip right in preview, then lock it and render once at full quality.

PREVIEW RENDER

- Fast, so you can review quickly
- Smaller file
- CRF around 23, lower quality is fine
- Use this for every review pass

HQ RENDER

- Only for approved clips
- CRF 14 to 16, clean and sharp
- Clean audio, final pixel format
- One render per locked clip

the two commands

```
# preview, fast and rough
npx remotion render src/index.ts Clip3 out/clip3_preview.mp4 --crf=23

# final HQ, only after approval
npx remotion render src/index.ts Clip3 out/clip3_HQ.mp4 --crf=14 --pixel-format=yuv420p
```

NAME EVERY FILE

Use names that tell you what the file is: `clip3_split_preview_v4.mp4` and `clip3_split_HQ.mp4`. Six months later you will still know which one was final.

THE RENDER REPORT YOU ASK FOR

```
duration: 6.0s
resolution: 1080x1920
captions: exact, locked
output: edited_outputs/.../clip3_HQ.mp4
assumptions: none, all confirmed
```

DO THIS NOW Always ask the agent for a render report with duration, resolution, captions used, output path, and any assumptions it made. If it assumed something, you catch it before posting.

REUSABLE SHAPE

Your 4 clip reel blueprint

This is the exact shape you can reuse for almost every tutorial reel. Four clips, four jobs. Build them one at a time, then merge at the very end.

1. **Clip 1 / Hook**
   Identity plus a reason to keep watching. Example: "Day 3 of editing reels with an AI agent." Pair it with one visual number on screen.

2. **Clip 2 / Explanation**
   State the idea in one plain line. Example: an AI coding agent can edit and animate the video for you because the video is code.

3. **Clip 3 / Proof**
   Show the real demo. A split screen works well here: your face on top, the actual Remotion edit on the bottom.

4. **Clip 4 / CTA**
   Tell them exactly what to DM and what they get for it. Keep the keyword readable and away from the Instagram buttons.

> **Why this works.** The viewer gets a promise, then an explanation, then proof, then a clear next action. That is the shortest honest path from a view to a follow or a DM. No trick, just order.

DO THIS NOW

Before you touch the agent, write the one sentence job for each of the four clips. If a clip does not have a clear job, cut it. Three sharp clips beat five blurry ones.

PREVIEW LOOKS OFF

Common mistakes and fixes

Use this page when your preview looks bad but you cannot name why. Find the symptom on the left, do the fix on the right, render one more preview.

Caption mismatch. The words on screen are not what you said.

Fix the transcript first, then re edit. Technical words like Remotion, Codex, and your DM keyword get misheard the most. Read the captions out loud once before approving.

Animation covers your face. The effect sits on your eyes or mouth.

Move it to open space or send it behind you. Eyes and mouth must stay readable. Tell the agent: keep the face clear, place the graphic in the empty area.

It looks childish. Cartoon rockets, fire, stick figures.

Replace cartoons with a dark grid, motion paths, keyframe dots, procedural particles, and kinetic typography. For AI content, mature motion beats cute every time.

Too subtle. Nothing grabs the eye.

Add one bigger centerpiece: an orb, a typography reveal, a timeline, or a clean layout transition. One strong centerpiece, not five weak effects.

CTA hidden. The DM line clashes with Instagram UI.

Move the CTA away from the bottom 250px and the right side buttons. If a viewer cannot read it, the edit failed even when the animation was great.

Agent changed too much. It rewrote things you liked.

Tell it exactly what to keep and what not to touch. Be literal: do not change the captions, the layout, or the audio. Only fix the number animation.

Files scattered. You cannot find the right version.

Force every preview and report into `edited_outputs/project_name/`. Never let the agent drop files randomly across the folder.

THE LAST GATE

Final reel checklist before posting

Run this before the reel goes to Instagram. If any single line fails, fix it before you post. This is the difference between looks fine and looks finished.

1. **Hook clear in 1 second**
   The viewer knows why to keep watching almost instantly.
2. **Captions exact**
   No wrong technical words, no emotion where you meant Remotion.
3. **Face visible**
   No animation covering your eyes or mouth at any point.
4. **Proof visible**
   The demo is not too small and not hidden behind the UI.
5. **CTA readable**
   The DM keyword is obvious and clear of the Instagram buttons.
6. **Version locked**
   Approved previews were rendered as HQ individual clips.
7. **Final merge checked**
   No clip starts or ends awkwardly, and the audio is clean throughout.

DO THIS NOW

Do not ask for the final merge until you have watched every HQ clip on its own. The merge is the last step, not the cleanup step.

FROM ZERO, IN FIVE DAYS

Your 5 day practice plan

One small output each day. Do not skip Day 0, it is the setup that makes the rest possible. By the last day you have shipped a real reel, not just read about one.

0 **Day 0 / Setup**

Work through Part 1. Install Node, the terminal basics, VS Code, your agent, and make the Hello World project. Stop once Studio opens.

1 **Day 1 / Caption only clip**

One face cam clip. Exact captions, one highlighted word, safe placement near the chin. Nothing else.

2 **Day 2 / Object trail hook**

Make a ball, hand, or object create one clean visual. One animation, done well.

3 **Day 3 / Split screen proof**

Face cam on top, a mature motion canvas on the bottom. Use the 36/64 ratio.

4 **Day 4 / CTA ending**

A guide card, a DM keyword, and a safe CTA placement. Then merge the four clips into one reel.

> **Do not skip review.** Every day, watch the preview as a viewer, not as the person who built it. Ask one question: did I understand the idea without knowing the behind the scenes context? If not, fix that before moving on.

WHEN YOU FORGET

Quick command cheat sheet

Copy these when you forget what to run. Commands can change with the Remotion version and your project setup, so keep a COMMANDS.md in each project with the exact ones that worked.

basics

```
# create a new project
npx create-video@latest

# open Remotion Studio to preview
npx remotion studio

# render a quick preview
npx remotion render src/index.ts MyComp out/preview.mp4 --crf=23
```

render

```
# render high quality, only when approved
npx remotion render src/index.ts MyComp out/final_HQ.mp4 --crf=14 --pixel-format=yuv420p

# render a single still frame if you need one
npx remotion still src/index.ts MyComp out/frame.png --frame=120
```

DO THIS NOW

If a command fails, do not guess. Open the project package.json, find the scripts, and tell the agent: this command failed, use the correct local command for this project.

THE SOURCE OF TRUTH

Source notes and further learning

This guide is the practical layer: what to ask, what to check, how to judge a preview. The official Remotion docs are the source of truth for exact APIs and version changes. Use both.

START HERE

- **Homepage** `remotion.dev`
- **Fundamentals** `remotion.dev/docs/the-fundamentals`
- **Assets and staticFile** `remotion.dev/docs/assets`
- **Rendering** `remotion.dev/docs/render`

WHEN YOU ANIMATE

- **API overview** `remotion.dev/docs/api`
- **Animating properties** `remotion.dev/docs/animating-properties`
- **interpolate and spring** `/docs/interpolate`, `/docs/spring`
- **Sequence and Captions** `/docs/sequence`, `/docs/captions`

How to use this PDF with the docs. When you know what you want to build, open this PDF and write the prompt. When you need an exact API detail or a render flag, open the docs. The agent can read the docs for you, just point it at the right page. Player and Lambda live at `/docs/player` and `/docs/lambda` when you scale up.

YOUR NEXT ACTION

Build one clip today

Do not wait until you understand all of Remotion. Nobody does. Take one raw clip, add exact captions, one clean animation, one preview render, and one report. That is a real reel started.

TINY FIRST PROMPT / COPY THIS

```
Edit only clip1. Add exact captions and keep them near my chin. Add one clean motion graphic in the empty space, not on my face. Save a preview and a short report in edited_outputs.
```

AI WITH MUDIT

Use AI like a builder, not like a shortcut.

DM "REMOTION"